A Guarded Orbit Presentation of iPRA

A further reduction of the ConciseIPRA checker, with an explicit equivalence boundary

Research report prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We give a further, small reduction of the current ConciseIPRA presentation. Elimination selects its major premise from its source term, and introduction selects cases or equation progress from the root of its matrix. These two reductions already exist separately. They do not compose inside the strictly guarded calculus: an assumption $0 = t$ cannot there yield $t = 0$. The new argument repairs that failure using the intermediate orbit formula $D_{r,y}(y = I(y, t; u.Su))$. It supplies the one closure needed to recover zero-source equality symmetry. Stable-left numerical cases then suffice to complete the bootstrap, and the original guarded formula language is retained for public statements. We prove that this presentation is equiconsistent and whole-domain bi-interpretable with full first-order intuitionistic primitive recursive arithmetic, with actual equality and identity numerical composites.

Three complete Python programs improve the separate attained coordinates from 259 tokens, 283 AST nodes and 526 normalized bytes to 258, 281 and 525. These are different programs. The compact comparison assumes canonical public inputs. A supplied checker that also validates the public grammar is 1205 bytes, compared with 1208 for the prior guarded interface and 1109 for the prior scalar interface. Thus this is a small positive result under the repository's canonical-input convention, not an absolute simplest theory or a new minimum for every strict interface. The new metatheorems are written proofs; no new Lean or Rocq verification is claimed.

The mathematical contribution. Intermediate formulas can be more expressive than the public language without changing its theory. Here a single family of iterator-orbit formulas makes two otherwise incompatible rule reductions work together. The statement of the theory includes this distinction; it is not an omitted validation cost.

Contents

1 Result, reference point, and meaning of simpler **3**

2 The complete presentation **3**
2.1 Numerical terms and finite descriptions . . . 3
2.2 Two explicit formula grammars . . . 4
2.3 Axioms and inference rules . . . 4
2.4 Ordinary reading and immediate soundness . . . 5

3 The orbit bootstrap for guarded source dispatch **5**
3.1 The restriction that prevents a naive composition . . . 6
3.2 Safe equality and padding before zero-source symmetry . . . 6
3.3 Unconditional tests . . . 7
3.4 The administrative orbit supplies the missing induction . . . 8
3.5 Recovering the old guarded calculus . . . 8
3.6 Administrative soundness and the public theorem . . . 9

4 Full interpretations and both composites **9**
4.1 Padding and arbitrary numerical cases after the bootstrap . . . 9
4.2 Contextual equation induction without an eigenvariable error . . . 10
4.3 Translation of the current Q/D language . . . 11
4.4 Composition with the inherited arithmetic bridge . . . 12

5 Complete checker sizes and the input contract **12**
5.1 What is included in each measurement . . . 12
5.2 Canonical public inputs versus a strict interface . . . 13
5.3 The entire 525-byte normalized module . . . 14

6 Validation, reproducibility, and evidence limits **14**
6.1 A correction to the current README's substitution wording . . . 15

7 Further research questions **15**

Attribution and claim ledger **16**

1 Result, reference point, and meaning of simpler

The reference is the user's ProveIt repository at commit `1517d9d79ce33f631e0f82960c849427cbba23bf`, under `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA` [1]. At that commit the Lean kernel already has equational transport and direct Q progress; literal-zero source elimination is also proved in Lean. The written packages additionally contain guarded, root-determined introduction. None of these individual changes is claimed as new here. The current README lists twenty-eight research packages, and reports separate checker minima for the `source_dispatch` family.

The new result is the *compatibility proof with administrative orbits*. It is necessary: simply intersecting the two reduced rule systems does not preserve equality reasoning. The obstruction and its repair are proved below. The numerical syntax, scalar arithmetic, logical encodings, and existing iPRA bridge are inherited and explicitly attributed.

Theorem 1.1 (Public theory and numerical composites)**.** *Let $\mathcal{G}$ be the public language of Definition 2.1, and permit proofs in the administrative calculus $\mathcal{U}_{\neq 0}$ of Section 2. For every finite context Γ and conclusion A in $\mathcal{G}$, this calculus derives exactly the sequents of the inherited guarded presentation. There are effective interpretations between its ordinary realization and full first-order* iPRA*, and each formula is provably equivalent to its round trip. Both numerical domains are whole, equality is actual equality, and both numerical composites have graph $x = y$. In particular, the two theories are equiconsistent.*

Here "full first-order" describes the available logic, including arbitrary quantified formulas. It does not mean full induction. On the iPRA side, induction is restricted to quantifier-free formulas; the arithmetic progress rule below applies only to literal equations.

The comparison is deliberately qualified. There is no proved universal lower bound of 259 tokens, 283 nodes, or 526 bytes. Those are attained coordinates of inspected programs. The smaller programs below recognize exactly the intended theorems on canonical public inputs, while allowing broader intermediate syntax. The cost of validating canonical inputs is reported separately and also included in one standalone strict program. The latter does not beat the 1109-byte old scalar interface. A claim of an unqualified new global minimum would therefore be unsupported.

2 The complete presentation

2.1 Numerical terms and finite descriptions

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u is bound in h. The counter n and initializer a are outside that binder. Other variables in h are passive parameters. Substitution is capture-free, and bound variables may be renamed. Write $1 = S0$, $2 = S1$, and

$$C(v, a, b) := I(v, a; u.b),$$

with u fresh for b. These are abbreviations, not extra constructors. The finite descriptions can equivalently be expanded to ordinary first-order function symbols, one for each fixed previously constructed iterator body. No variable ranges over functions or over descriptions, and no uniform total evaluator is postulated internally.

2.2 Two explicit formula grammars

The administrative grammar is

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

Q binds y; D binds the distinct variables x, y. A *literal equation* is a formula whose outermost constructor is equality. Every other administrative formula is called compound. Retained contexts are finite reusable lists of administrative formulas.

Definition 2.1 (Public formulas)**.** The public language $\mathcal{G}$ is the least grammar satisfying:

1. Every equation $s = t$ belongs to $\mathcal{G}$.
2. $D_{x,y}B$ belongs to $\mathcal{G}$ when B belongs to $\mathcal{G}$ and is compound.
3. An atomic matrix immediately below D is allowed exactly when it has the shape

$$D_{x,y}\big(I(y, 0; u.h) = I(y, 0; u.k)\big).$$

   The two state binders are independent, and h, k may contain their state, x, y, and passive parameters.

There is no public Q constructor. It remains available in proofs. The designated contradiction is the public equation $0 = 1$.

In particular, the public language is exactly the earlier guarded language, not a newly enlarged language with an additional orbit clause. The orbit formulas used later occur only inside proofs. Every public guarded atomic matrix has a provable zero instance, since both sides compute to zero. This syntactic fact is the reason a single introduction scheme suffices.

2.3 Axioms and inference rules

The two computation schemas are available under every context:

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0,IS}$$

The remaining rules are precisely the following. No unlisted logical, equality, successor, or induction axiom is primitive.

Hypothesis. If $A \in \Gamma$, then $\Gamma \vdash A$.

Source-selected elimination. For an administrative formula template $F(v)$, define

$$M(s, t, F) = \begin{cases} Q_v F(v), & s \text{ is literally } 0, \\ s = t, & s \text{ is not literally } 0. \end{cases}$$

The rule is

$$\frac{\Gamma \vdash M(s, t, F) \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{E}$$

Thus literal zero selects Q elimination and every other syntactic source selects equality transport. An iterator term denoting zero is a permitted equality source. This is a test of syntax, never a test of its numerical value.

D elimination.

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, Q_yB(b,y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

The eigenvariable b is fresh for Γ, A and the passive parameters of B. Its local assumption is discharged.

Root-determined D introduction. Put

$$\eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases}$$

Then

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x,y)}. \tag{DI}$$

Here z is fresh for Γ, t and the passive parameters; the local assumption is discharged. The witness t is any numerical term and is chosen before introducing z. The certificate has no separate case/progress choice. At an equation the rule is progress; at a compound matrix it is cases. Arbitrary unguarded atomic matrices are allowed as intermediate formulas, including the orbit matrix used in the proof.

Structural conventions. Weakening, exchange, contraction, renaming, and ordinary-assumption cut are admissible proof transformations. They are not additional checker instructions. Substitution into a restricted derivation is used before the bootstrap only when all substituted transport sources remain syntactically nonzero. The main proof does not presuppose unrestricted substitution.

2.4 Ordinary reading and immediate soundness

On public formulas, read equality as actual numerical equality and

$$Q_yB \;:=\; B(0) \to \forall y B(y), \qquad D_{x,y}B \;:=\; \exists x(B(x,0) \to \forall y B(x,y)). \tag{R}$$

These readings are also sound for all the administrative rules in the inherited scalar theory. Atomic DI follows by equation induction, compound DI follows by numerical coverage, and DE is existential elimination. The source-selected rule is an allowed equality or Q elimination. Thus every administrative proof maps to an old scalar proof under the ordinary reading. What is *not* asserted is completeness for that reading on every administrative formula. The completeness and bi-interpretation claims have the explicitly stated public domain $\mathcal{G}$.

For one noncompleteness test later, a second sound reading of the administrative language is useful: atomic D means witnessed progress, $\exists x \forall y(E(x,y) \to E(x,Sy))$; compound D and every Q keep the anchored reading in (R). This mixed reading agrees with (R) on public guarded formulas. It is an auxiliary semantic argument, not an extra primitive rule or an implicit change in the public interpretation.

3 The orbit bootstrap for guarded source dispatch

The two existing reductions do not compose within the strictly guarded calculus. They do compose when the proof language retains the administrative orbit formulas described below. The distinction between public statements and intermediate proof formulas is essential.

Let $\mathcal{G}$ be the existing guarded calculus. Its terms are $x, 0, St, I(n, a; u.h)$, and its public formulas are equations and $D_{x,y}B$. An atomic matrix immediately below D has the shape

$$I(y, 0; u.h) = I(y, 0; u.k),$$

while a compound matrix is unrestricted within this recursive grammar. Write $C(v, a, b) = I(v, a; u.b)$, with unused u, and $G_v(s = t) \equiv C(v, 0, s) = C(v, 0, t)$.

For the intermediate argument, enlarge the grammar by also admitting the *orbit matrices*

$$O_a(y) \equiv y = I(y, a; u.Su), \tag{O}$$

where a is independent of y (it may contain the existential coordinate and passive parameters). Call this enlarged grammar $\mathcal{G}^+$. The primitive rules are those of the guarded calculus: computation, clause use, D elimination, and root-determined introduction (progress for atomic matrices, cases for compound matrices). Restrict transport to a source term that is not literally 0. Call this restricted calculus $\mathcal{R}$. No additional inference instruction is introduced by (O).

Contexts consist of reusable clauses $\langle A; y.B\rangle$, interpreted as $A \to \forall y B$. An ordinary retained assumption is a constant clause, as in the inherited checker. In what follows, all introduced D matrices are guarded, compound, or of form (O).

3.1 The restriction that prevents a naive composition

Call an equation *zero-left* if its left term is literally 0; call all other equations and all compound formulas regular. If the source s of a transport is not literally zero, a zero-left minor cannot become regular: if $p(s)$ is literally 0, the numerical template p must be the constant 0 template, so $p(t)$ is also literally 0.

In the strictly guarded calculus, both endpoints of every atomic clause opened by D elimination are regular. Every atomic introduction premise and its local assumption are also regular, since their left terms have an I head. Compound cases have compound endpoints. Consequently a retained assumption $0 = x$, for a fresh variable x, can be erased from every proof of a regular conclusion in that calculus. A simultaneous induction on the proof tree and its regular clause uses proves this assertion: a regular transport conclusion has regular premises; a regular clause conclusion uses a regular antecedent; the computation and binder rules preserve the invariant. The only added zero-left constant clause cannot be used on a path to a regular conclusion.

Thus $0 = x \vdash x = 0$ is not derivable there: otherwise $x = 0$ would be a theorem, contradicting the standard natural-number interpretation. This is why one cannot simply quote the two previous admissibility theorems in succession. In the present proof, orbit elimination has the nonregular antecedent $0 = I(0, a; u.Su)$ and can produce regular successor instances; it is exactly the missing bridge.

3.2 Safe equality and padding before zero-source symmetry

Weakening, renaming, and ordinary-assumption cut are admissible. A numerical substitution is admissible for a particular derivation if it leaves every primitive transport source syntactically nonzero. No unrestricted substitution into a restricted derivation is assumed.

For every r, let $b_r = I(0, r; u.0)$. The computation $b_r = r$, used as both major and minor of transport with motive $v = r$, proves $r = r$. From $p = q$ with $p \not\equiv 0$, the same rule gives symmetry $q = p$ and congruence $H(p) = H(q)$. Transitivity through a syntactically nonzero middle term is immediate.

Computation equations may therefore be reversed as equations; using the reverse as a transport major still requires its new source to be syntactically nonzero.

We also retain the inherited safe zero-join lemma:

$$p, q \not\equiv 0, \quad p = 0, \quad q = 0 \quad \Longrightarrow \quad p = q. \tag{ZJ}$$

For completeness, put $z_0 = I(0, 0; u.0)$. From $b_p = b_p$ and $p = 0$, transport in $I(0, v; u.0) = b_p$ gives $z_0 = b_p$. Reverse at source z_0, then use $b_p = p$ at source b_p to obtain $p = z_0$. Obtain $q = z_0$ in the same way, reverse it at source q, and compose through z_0. All sources are p, q, z_0, b_p, b_q, which are nonzero syntax.

A restricted lifting lemma. Suppose $p \not\equiv 0$, and $a, b \not\equiv 0$ have proved equations $a = p$ and $b = q$. From $p = q$ one can derive $a = b$ without zero-source transport. First compose through p to get $a = q$. If $q \not\equiv 0$, reverse $b = q$ and compose through q. If $q \equiv 0$, use (ZJ). Conversely $a = b$ transports forward along $a = p$ and $b = q$ to $p = q$.

In particular, for every equation $E \equiv p = q$ whose left term is not literally zero,

$$E \dashv\vdash G_{Sz}E, \qquad \vdash G_0E.$$

The forward implication uses the lifting lemma with the two selector computation equations. The reverse implication only contracts iterator sources and in fact is valid for arbitrary E. Define

$$L(E) = D_{r,v}G_vE,$$

with unused fresh r, v. For such a regular equation E, progress introduction, ignoring its local hypothesis, proves $E \vdash L(E)$. Elimination, the theorem G_0E, and use at 1 prove $L(E) \vdash E$.

Numerical cases sufficient for the bootstrap. Cases for compound matrices need no transport: introduce a dummy-witness D, open its clause, and use the supplied zero case at the target. Consequently cases is also available for any equation template $E(y)$ whose left side is headed by I or S. All its instances are regular, so pad the two branches by L, apply compound cases, and unpad. This restricted cases theorem suffices for every helper below. In particular, we do not invoke cases with an arbitrary zero-left equation as a retained assumption.

3.3 Unconditional tests

Define fixed finite terms

$$\begin{aligned} Z(v) &= C(v, 1, 0), & K(v) &= C(v, 0, 1), & H(v) &= C(v, 1, 2), \\ N(n) &= I(n, 0; u.Z(u)), & R(n) &= I(n, 0; u.H(u)), & L_1(n) &= N(R(n)). \end{aligned}$$

(The term L_1 is unrelated to the padding formula L.) The inherited singleton-test calculation works with the restricted cases just proved:

$$H(v) = SK(v), \quad R(1) = 1, \quad R(SSv) = 2, \quad N(1) = 1, \quad N(2) = 0,$$

$$L_1(1) = 1, \qquad L_1(SSv) = 0, \qquad L_1(Sv) = Z(v). \tag{T}$$

Here are the details relevant to the restriction. For $H(v) = SK(v)$, the zero and successor branches reduce to the nonzero numerals $1, 2$; join their endpoints through those numerals and apply restricted cases, whose left side is I-headed. The values of N and $R(1)$ are finite forward computations. From

$R(Sv) = H(R(v)) = SK(R(v))$, rewrite the argument of $R(SSv) = H(R(Sv))$ using the I-headed source $R(Sv)$ and compute H at a successor to get 2. This gives the values of L_1. For the last equation in (T), the zero branch joins through 1 and the successor branch uses (ZJ) on the two I-headed terms that are zero. Its left side $L_1(Sv)$ is again I-headed, so restricted cases applies.

For the filter, use cases on the *reversed* equation

$$C(Z(v), v, 0) = v. \tag{F}$$

Its left side is always I-headed. The zero branch is the forward calculation $C(Z(0), 0, 0) = C(1, 0, 0) = 0$. The successor branch is $C(Z(Sv), Sv, 0) = C(0, Sv, 0) = Sv$. All replacements use iterator sources. After cases has proved (F) at the arbitrary target t, safe symmetry at its I-headed source gives

$$t = C(Z(t), t, 0). \tag{F'}$$

All test lemmas are established in the empty context and only then weakened into a context containing a zero-left assumption.

3.4 The administrative orbit supplies the missing induction

For a fixed term t put $A_t(n) = I(n, t; u.Su)$. The formula

$$D_{r,y}(y = A_t(y)) \tag{OC}$$

with unused r is derivable in $\mathcal{R}$, without a base hypothesis. For progress introduction retain $z = A_t(z)$. Its source z is a fresh variable, hence syntactically nonzero. Successor congruence gives $Sz = SA_t(z)$. Computation gives $A_t(Sz) = SA_t(z)$; reverse this equation at its iterator source and compose through the successor term $SA_t(z)$. Thus $Sz = A_t(Sz)$, exactly the required progress premise. This derivation never substitutes 0 for the variable-source transport.

Now retain $0 = t$. If $t \equiv 0$, symmetry is reflexivity. Otherwise t is a permitted source. Reverse the computation $A_t(0) = t$ and use $t = A_t(0)$ at source t in the assumption $0 = t$ to get $0 = A_t(0)$. Open (OC), supply that base, and specialize its clause at 1. This gives $1 = A_t(1)$. Forward computation gives $A_t(1) = St$, so

$$1 = St.$$

Transport inside L_1 at source 1, then contract the I-headed sources using (T), obtaining $1 = Z(t)$. Safe symmetry at source 1 gives $Z(t) = 1$. Apply this equation at its I-headed source to (F'), then compute $C(1, t, 0) = 0$. The result is

$$0 = t \ \vdash_{\mathcal{R}} \ t = 0. \tag{ZS}$$

No unrestricted numerical substitution or contextual cases principle was used. The only new intermediate binder was (OC).

3.5 Recovering the old guarded calculus

From (ZS), safe symmetry, and (ZJ), obtain full symmetry, transitivity, and numerical congruence exactly as in the existing source-dispatch argument. For transitivity through literal zero, use (ZJ) if both endpoints are nonzero syntax; otherwise one premise is already the conclusion. For congruence along $0 = t$, use reflexivity when t is literally 0; otherwise reverse to $t = 0$, apply safe congruence there, and reverse the result. This derives unrestricted transport in a literal equation by joining the two endpoint congruences.

Full transport through a $\mathcal{G}$ formula is now proved by external induction on its binder count. In a compound-matrix D, use its case introduction, transporting the base backwards, specializing the old clause, and transporting the successor forwards by the smaller-formula induction hypothesis. In a guarded atomic D, its source zero instance is an unconditional theorem; hence the opened source clause supplies its successor instance. Transport that literal equation forward, and introduce the target by atomic progress, ignoring its local hypothesis. For an orbit D in $\mathcal{G}^+$, the target is already a theorem by the derivation of (OC). This also establishes full transport throughout $\mathcal{G}^+$ if that enlarged public grammar is desired.

Consequently every proof in the inherited guarded calculus translates to $\mathcal{R}$, with the same public context and conclusion. All nontransport rules are copied, and every old transport is replaced by the preceding derived rule. The known translations and both formula composites between the original scalar core and $\mathcal{G}$ therefore remain available. The numerical domain, terms, and actual equality are unchanged.

3.6 Administrative soundness and the public theorem

The compact combined checker admits more intermediate syntax than $\mathcal{G}^+$. Use the already established mixed administrative reading: an atomic D matrix has the progressive meaning $\exists x \forall y (E(x,y) \to E(x,Sy))$, and a compound matrix has the anchored meaning $\exists x (B(x,0) \to \forall y B(x,y))$. Clauses keep the anchored reading. Equation induction in iPRA makes atomic elimination sound; introduction and the other rules are sound directly. Restricting transport sources preserves this soundness.

On a public guarded atomic matrix, the base is provable, so the progressive and anchored readings agree. On an orbit matrix, both readings are unconditional theorems by the displayed progress proof and equation induction; their agreement does not rely on a true base. Thus both possible choices of public grammar, $\mathcal{G}$ and $\mathcal{G}^+$, have the ordinary guarded meaning on their public statements. Administrative proofs are conservative there by soundness in iPRA and the completeness translation just proved.

The resulting precise statement is: for canonical guarded end sequents, the combined administrative checker recognizes exactly the guarded calculus's consequences, and its ordinary realization is equiconsistent and whole-domain bi-interpretable with iPRA. This is not an equivalence claim for every unrestricted administrative formula. It is also not a claim that the compact checker validates the canonical public grammar; if that is required, its parser must be included in the size accounting.

4 Full interpretations and both composites

This section supplies the remaining link from the preceding orbit bootstrap to the actual current native scalar core. It also records which portion of the arithmetic bridge is inherited. Write $A \dashv\vdash B$ for uniform interderivability under arbitrary additional retained assumptions.

4.1 Padding and arbitrary numerical cases after the bootstrap

For an equation $E = (s = t)$ put

$$G_v E = (C(v,0,s) = C(v,0,t)), \qquad L(E) = D_{r,v} G_v E,$$

where r, v are fresh and r is unused. Full equational transport, now derived by the orbit bootstrap, proves

$$\vdash G_0 E, \quad G_{Sz} E \dashv\vdash E, \quad L(E) \dashv\vdash E \tag{P}$$

for *every* equation, including zero-left equations. Let $P(A) = L(A)$ for an equation and $P(A) = A$ for a public compound formula. Thus $P(A)$ is always compound and $P(A) \dashv\vdash A$.

Compound numerical cases is already available from DI and DE. Applying it to $L(E(y))$ gives numerical cases for arbitrary equations as well. There is still a freshness condition: the context of the two case branches must be independent of the case variable. In particular, one must not perform cases on y while retaining an unprocessed assumption $E(y)$.

4.2 Contextual equation induction without an eigenvariable error

We first specify the proof subsystem in which the following structural operations are made. Let $\mathcal{G}^+$ be the guarded language enlarged by the orbit matrices $y = I(y, a; u.Su)$, with a independent of y. Use these as object formulas, with their anchored Q clauses as administrative context resources. All proofs constructed in the orbit bootstrap lie in this subsystem. Full transport has now been derived throughout $\mathcal{G}^+$: the guarded and compound cases were proved there, and an orbit D target is already a theorem by its progress proof.

Temporarily allow this derived full-transport rule as an inference node. Numerical substitution is admissible in that derived calculus by induction on derivations, renaming eigenvariables first. The syntax of $\mathcal{G}^+$ is preserved by substitution, the computation and binder rules retain their forms, and full transport has no literal-source restriction to invalidate. After substitution, replace every full-transport node by its restricted derivation. This produces a derivation in $\mathcal{U}_{\neq 0}$. Thus substitutions into the constructed $\mathcal{G}^+$ proofs, including their administrative orbit steps, are justified. This is a statement about the specified clause subsystem, not about arbitrary administrative derivations in the entire language of $\mathcal{U}_{\neq 0}$.

For equations $U = (s = t)$ and $V = (r = q)$ define

$$M_w(U, V) = (C(w, s, r) = C(w, t, q)), \qquad U \Rightarrow_g V = D_{a,w} L(M_w(U, V)),$$

with fresh unused a. The outer matrix is compound. The selector computations and (P) identify its zero endpoint with U and every successor endpoint with V. Consequently this macro admits implication introduction and elimination between equations. Introduction is compound case DI; elimination opens its clause and uses it at 1.

Lemma 4.1 (Contextual equation induction)**.** *For premise derivations in the $\mathcal{G}^+$ subsystem just described, the following rule is admissible:*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)},$$

where E is a literal equation and z is fresh for the fixed data.

Proof. Let $J(y) = G_y E(y)$. We seek the guarded progress premise $\Gamma, J(z) \vdash J(Sz)$. Use cases on the *compound* formula

$$\Phi(y) = J(y) \Rightarrow_g E(Sy)$$

in the fixed context Γ. At zero, ignore $J(0)$, use the supplied base $E(0)$ and the step at 0. At a successor Sw, unguard $J(Sw)$ to $E(Sw)$ and use the step at Sw. The two branches have no retained assumption containing their case variable. Cases therefore gives $\Gamma \vdash \Phi(z)$. Apply it under $J(z)$ and guard the result to obtain $J(Sz)$.

Atomic DI yields $D_{r,y} J(y)$ with unused witness coordinate. Its base is a theorem, so DE and specialization supply $J(Sw)$, hence $E(Sw)$. Finally numerical cases for E, using the original base, yields

$E(t)$. The occurrences of the supplied step at 0 and Sw use substitution in the derived full-transport $\mathcal{G}^+$ calculus, followed by expansion of its transport nodes as specified above. No direct unsafe substitution into a restricted primitive derivation is asserted. □

This lemma suffices for the recursive translation below: that translation constructs only guarded object formulas, orbit intermediates, and their anchored clauses. Its output therefore stays in $\mathcal{G}^+$, so every recursive induction-step premise meets the stated hypothesis. After the two interpretation theorems are proved, equation induction is also an admissible rule of the full public consequence relation. It is not being assumed in that broader form to establish those theorems.

4.3 Translation of the current Q/D language

Let $\mathcal{N}$ be the current native scalar core. Its terms are the same terms, and its formulas are equations, Q_yB, and $D_{x,y}B$. Define T recursively by

$$\begin{aligned} T(s=t) &= (s=t), \\ T(Q_yB) &= D_{r,y}P(TB), \quad r \text{ fresh and unused}, \\ T(D_{x,y}B) &= D_{x,y}P(TB). \end{aligned} \tag{T}$$

Capture-free renaming is understood. Every output belongs to $\mathcal{G}$: every translated matrix is compound, and the atomic matrices introduced inside L have the required zero guard. T leaves every numerical term unchanged and commutes with numerical substitution up to bound renaming. Let $J : \mathcal{G} \to \mathcal{N}$ be literal inclusion.

Proposition 4.2 (Both derivation translations)**.** *For native* Γ, A *and public* Δ, B*,*

$$\Gamma \vdash_{\mathcal{N}} A \implies T\Gamma \vdash_{\mathcal{U}_{\neq 0}} TA, \qquad \Delta \vdash_{\mathcal{U}_{\neq 0}} B \implies J\Delta \vdash_{\mathcal{N}} JB.$$

Proof. For the second implication, read each administrative rule as the inherited native rule. Its two introduction cases are allowed witnessed DI rules, which the current native core derives; this is the upstream `NativeInherited.lean` equivalence. Restricted transport is a native derived rule. The administrative Q is exactly native Q for this direction.

For the first implication, translate by induction on the native derivation. Computation and hypothesis are unchanged. Equational transport is available from the orbit bootstrap. A translated Q is a dummy-witness D over $P(TB)$; opening it supplies a clause whose endpoints are interderivable with those of TB. This simulates QE by padding its base, using the clause at the same term, and unpadding the result.

For DE, open the translated D at a fresh witness b. Its clause has antecedent $P(TB)(b, 0)$ and consequent $P(TB)(b, y)$. Derive the entire translated hypothesis $T(Q_yB(b, y)) = D_{r,y}P(TB(b, y))$ from this clause: introduce the dummy-witness binder by compound case DI and, under its local base, use the opened clause at Sz. Cut that formula into the recursively translated minor, then close the outer elimination. This treats arbitrary uses of the native Q hypothesis, not merely its uses as a QE major. The witness is absent from the unchanged final conclusion. For native case DI, introduce the translated binder by compound case DI: unpad its base, apply the translated premise, and pad its successor target. For native direct equation progress, use Lemma 4.1 under the retained base to obtain every successor instance, then introduce the dummy-witness translated Q by compound case DI. These are effective finite derivation transformations within the $\mathcal{G}^+$ clause subsystem. Each use of structural substitution is made in its derived full-transport calculus and then expanded, as specified above. □

Proposition 4.3 (Both formula composites)**.** *For every native formula A and public formula B,*

$$A \dashv\vdash_{\mathcal{N}} JT(A), \qquad B \dashv\vdash_{\mathcal{U}_{\neq 0}} TJ(B).$$

The equivalences are uniform under retained native or public contexts, respectively.

Proof. In the native calculus, padding is equivalent to its argument. A dummy-witness D is equivalent to Q, and arbitrary native D matrices respect uniform interderivability. Structural induction proves the first equivalence.

For the second, equations are unchanged. A compound public matrix respects interderivability by opening the source binder, introducing the target by case DI, converting its base backwards, using the source clause at a successor, and converting forwards. For a guarded atomic matrix $E(x, y)$, prove $D_{x,y}E \dashv\vdash D_{x,y}L(E)$. Forwards, open the source and use compound case DI after unpadding the target base. Backwards, $E(b, 0)$ is an unconditional theorem; pad it, use the opened clause to obtain $L(E(b, Sz))$, and unpad. Atomic progress DI introduces DE while ignoring its local induction hypothesis. Close DE. This establishes the atomic case of the structural induction, and proves the second equivalence. □

4.4 Composition with the inherited arithmetic bridge

The current repository already proves whole-domain bi-interpretability of the native scalar core and the usual primitive-recursive-function language in Lean. The exact exported results are `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` in `Lean/ConciseIPRA/NativeBiInterpretation.lean` [4]. The ordinary scalar theorem is `biInterp: BiInterpTeqAxPRAAx` in `BiInterpretation.lean` [5].

That bridge derives a predecessor, equality reflection into a numerical term, quantifier-free induction, pairing, and compilation of primitive recursion from scalar iteration. The exact attributed written arithmetic sources and the two theorem-export files are included under `reference/`. They are inherited dependencies, not discoveries in this report. The existing Lean build was not rerun here, and these copied export files are evidence of the upstream result, not a self-contained local Lean project. The new orbit argument has not been formalized in Lean or Rocq.

Compose that bridge with T and J. All term translations between the public and native languages are literally identity maps. The inherited scalar/iPRA term translations have internally proved identity composites. Proposition 4.3 supplies the new formula composites. Thus both resulting interpretations use the entire numerical domain and actual equality, and both composite isomorphisms have graph $x = y$. This proves Theorem 1.1, not merely mutual interpretability or coincidence of true closed numerical equations.

The contradiction stays $0 = 1$. The public/native translations preserve it literally, and the existing bridge identifies native inconsistency with iPRA proving $\bot$. Equiconsistency follows as an external syntactic theorem; no internal proof of iPRA's consistency is claimed.

5 Complete checker sizes and the input contract

5.1 What is included in each measurement

Each reported kernel is a complete standalone Python module importing only the standard JSON parser. It reads one JSON certificate from standard input and accepts exactly on normal exit with

output `ACCEPT` followed by a newline. Exceptions are rejection. No proof producer, translator, axiom database, or helper is imported by a checker.

The universal recognition statement uses the idealized unbounded integer-and-array semantics of the inherited sorted codec, together with the present derivation translations. Ordinary Python has finite memory and recursion limits, so it can reject sufficiently large valid certificates. Successful executions retain the stated soundness guarantee. The finite tests do not independently establish the inherited codec theorem.

We use the repository's normalization convention: remove genuine module, function, and class docstrings, apply `ast.unparse`, append one newline, and count significant tokenizer tokens, all AST nodes, and UTF-8 bytes. The runtime is CPython 3.12.14. Gzip level 9 with timestamp zero is reported for information, not optimized as the primary objective. The reference files were checked against their upstream Git blob hashes.

Complete program	Tokens	AST nodes	Bytes	Gzip-9
Old AST profile	263	283	607	348
New AST profile	262	**281**	606	348
Old byte profile	266	305	526	330
New byte profile	265	303	**525**	331
Old token profile	259	289	572	339
New token profile	**258**	287	571	339

The attained coordinate envelope therefore changes from $(259, 283, 526)$ to $(258, 281, 525)$, in token/node/byte order. Each triple combines coordinates from *different programs*; no listed program achieves all three. The byte-profile gzip grows by one byte. Raw-source and normalized-source gzip measurements are both recorded in the machine-readable metrics.

The dispatch change replaces the old independently selected mode test `len(a) % 2 == u` (or its token-profile counterpart) by `not len(a) % 2`. The source-dispatch restriction is retained. The small source edit is justified by the substantial derivability argument above; without the administrative orbit route, its apparent combination would have the obstruction proved earlier.

5.2 Canonical public inputs versus a strict interface

The compact programs do not enforce the public guarded grammar. Their theorem-recognition guarantee applies to its canonical encodings. This convention must be part of the comparison, rather than an uncounted front end added afterwards. Broader intermediate formulas are intentional, and are indispensable to the new bootstrap.

`code/strict_checker.py` includes the same goal validator as the earlier guarded package, coupled to the new kernel. It is a complete standalone module and uses the unchanged public grammar.

Complete strict interface	Tokens	AST nodes	Bytes	Gzip-9
Old scalar interface	450	521	1109	538
Old guarded interface	491	568	1208	574
New guarded interface	491	563	1205	572

The strict guarded interface improves by three bytes and five AST nodes against its matched predecessor. It remains 96 bytes larger than the old scalar interface. This is the principal practical limitation of calling the overall presentation "simpler". The result improves the established canonical-input frontier, not every possible complete-input contract.

5.3 The entire 525-byte normalized module

```
import json

def c(a, g=[[]], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d
    else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i
    (*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or not len(a) % 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

Here formula arities one, two, and three represent D, equality, and administrative Q, respectively. Term arities one and three represent successor and iteration. The opening operation uses the inherited sorted arity codec and its dummy slots; the exact protocol is supplied in `code/FORMAT.txt`, with the original codec justification under `reference/`. The local-scope AST and byte programs and the global-scope token program need separately numbered certificates.

The mode-free introduction does not remove the requirement that a witness be chosen before its eigenvariable becomes available. The local proof context must also be restored after checking a major premise. The new programs preserve these scope conventions; they do not exchange a shorter source for an unchecked freshness assumption.

6 Validation, reproducibility, and evidence limits

The archive contains a readable independent expansion, compact and strict checkers, named proof producers, serialized certificates, rejection controls, measurements, and source provenance. The executable evidence exercises the orbit bootstrap and the recovered equation rules. There were 22 test cases across five complete implementations, each run in ordinary Python and `python -O`: all 220 expected outcomes matched. Fourteen cases are positive canonical statements, one is a positive administrative orbit statement (deliberately rejected by the strict public-goal interface), and seven are negative controls. Four additional matched predecessor controls also passed. The precise counts and verdicts are in `evidence/validation.json` and `evidence/matched_baseline_controls.json`. The producer is not trusted by any kernel. Every generated proof is audited for primitive nonzero-source transport and root-determined introduction before serialization.

The generic named derivation of $0 = t \vdash t = 0$ contains 326 distinct proof nodes: 171 transport nodes, 122 computation nodes, and 10 introduction nodes among them. Exactly one introduction is the unguarded orbit step, and its progress hypothesis is used. The closed universal version has a 550-instruction local certificate. The independent guarded equation-induction example uses its induction hypothesis and has 256 instructions. Negative controls include the designated contradiction, an unallocated closed parameter, an unavailable introduction witness, malformed records, and an old atomic-case proof whose matrix is outside the public grammar.

The central distinction is between finite execution and universal proof. The report gives the universal derivations and interpretation composites. Certificates check concrete instances, scope

behavior, and rejection of invalid data. They are not a formal proof of every Python execution or a Lean/Rocq formalization of the metatheorem.

Reproduction from the extracted directory requires Python's standard library. Rebuilding the included PDF additionally requires a normal LaTeX installation with the packages named in the source:

```
python3 reproduce.py
pdflatex -interaction=nonstopmode -halt-on-error ipra_guarded_orbit.tex
pdflatex -interaction=nonstopmode -halt-on-error ipra_guarded_orbit.tex
```

No network fetch is required to run the supplied checker evidence or rebuild the PDF. The report's LaTeX source is a single file. Frozen earlier theorem exports are not claimed to build by themselves as a Lean project.

6.1 A correction to the current README's substitution wording

The current README says that substitution is "not admissible" in the literal-zero-source restricted calculus. The precise issue is that a direct rule-by-rule substitution can turn a permitted variable source into literal zero. Admissibility itself follows from its already proved equivalence:

$$ZPrf \longrightarrow NPrf \xrightarrow{\text{subst}} NPrf \longrightarrow ZPrf.$$

The first and last arrows are the two directions of `zprf_iff_nprf`. This observation does not justify a circular bootstrap: that equivalence must have been proved first. The new proof follows the same discipline and makes no unrestricted substitution assumption before the needed equality rules have been recovered.

7 Further research questions

1. **A smaller strict interface.** Can a single public syntax make the root-selected introduction complete while being simpler to validate than the current guarded grammar? The concrete next target is to beat the 1109-byte scalar strict comparator under the same complete-input contract.
2. **Mechanization.** Formalize the public/admin distinction and the stable-left cases lemma in Lean, then reuse the current `NativeZeroSource` arithmetic after replacing its general cases calls and its filter orientation. The orbit introduction should be the only genuinely new arithmetic bridge.
3. **Minimal intermediate language.** The completeness proof uses guarded formulas, their Q clauses, and the orbit family $y = I(y, a; u.Su)$. Is there a comparably short checker that enforces this smaller intermediate grammar, with a simpler universal soundness proof than the full arity codec?
4. **Proof-size cost.** Quantify the expansion caused by safe zero-join, guarded padding, and the implication needed for contextual induction. A smaller kernel can have longer certificates; those are independent optimization questions.
5. **Uniform full-language introduction.** Can the independent case/progress choice be eliminated while retaining every original administrative formula and its ordinary reading, with no new validator cost? The present result establishes completeness only on the stated public language, so this stronger problem remains open.

6. **A mathematical gain beyond one-byte refinements.** Can a different numerical basis remove a constructor or a binding operation without replacing it by a richer arithmetic axiom or more complex parser? The prior register, tied-seed, and zero-start results should be treated as constraints, not rediscovered as new proposals.

Attribution and claim ledger

The scalar term language, ordinary/native/iPRA bridge, source dispatch, safe zero-join, singleton test, guarded padding, and raw certificate codec come from the cited ProveIt research and formalization. The contribution here is the failure analysis for strictly guarded composition and its repair by administrative orbit formulas, including the restricted cases bootstrap and the resulting compatible checker family. The equality-safe filter is oriented in the reverse direction before the bootstrap.

The proved result is a bi-interpretable presentation and a small improvement under an explicit comparison convention. No literature-wide priority, global source-size lower bound, globally simplest axiomatization, shortest proofs, new proof-assistant verification, or internal consistency proof is claimed. The repository was read at a pinned commit; no remote changes were published.

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, current README at commit `1517d9d79ce33f631e0f82960c849427cbba23bf`. https://github.com/VladimirReshetnikov/ProveIt/blob/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[2] ProveIt, *A simpler elimination rule for intuitionistic primitive recursive arithmetic*, `Research/source_dispatch/`, same commit. Written proof, standalone checkers, and source provenance. https://github.com/VladimirReshetnikov/ProveIt/tree/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/source_dispatch.

[3] ProveIt, *A guarded core for intuitionistic primitive recursive arithmetic*, `Research/guarded_core/`, same commit. https://github.com/VladimirReshetnikov/ProveIt/tree/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/guarded_core.

[4] ProveIt, `NativeBiInterpretation.lean`, same commit. https://github.com/VladimirReshetnikov/ProveIt/blob/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeBiInterpretation.lean.

[5] ProveIt, `BiInterpretation.lean`, same commit. https://github.com/VladimirReshetnikov/ProveIt/blob/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/BiInterpretation.lean.

[6] ProveIt, `NativeZeroSource.lean`, same commit. https://github.com/VladimirReshetnikov/ProveIt/blob/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeZeroSource.lean.

[7] ProveIt, attributed scalar arithmetic proofs in `Research/certified_zero/reference/`, same commit: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`. Copies are supplied in this archive under the repository's MIT No Attribution license.